



C. F. G. S. Desarrollo de Aplicaciones Web - Distancia -

Examen de Programación de la 3ª Evaluación. 19-mayo-2026

“ *First, solve the problem. Then, write the code.* ” - John Johnson –

- 1) Se quiere desarrollar un sistema para gestionar setas recogidas en el monte y evaluar su idoneidad para el consumo. Programar una clase de nombre Seta, que tenga las siguientes propiedades (protected):

- String nombre
- String especie
- double peso (en gramos)

Tendrá los siguientes métodos:

- Métodos Getters y setters para las propiedades de la clase
- Método constructor por defecto que no haga nada.
- Método constructor al que se le pasen las tres propiedades de la clase y que valide que el peso sea mayor que 0, sino tiene que asignar los valores:

```
nombre = "Error"
especie = "Desconocida"
peso = 0
```

- 2) Programar una clase de nombre SetaComestible que herede de la clase Seta, y que tenga las siguientes propiedades (private):

- int nivelSabor (1 a 10)
- boolean venenosaLeve
- int diasConservacion (días desde la recolección)

Tendrá los siguientes métodos:

- Métodos Getters y setters para las propiedades de la clase
- Método constructor por defecto que no haga nada.
- Método constructor al que se le pasen las tres propiedades de la clase base. El constructor tiene que invocar al constructor de la clase de la que hereda. Al resto de las propiedades se le darán valores de manera aleatoria:

```
nivelSabor (de 1-10, ambos incluidos)
venenosaLeve (true/false)
diasConservacion (de 0-7, ambos incluidos)
```

- Método constructor al que se le pasen las seis propiedades de la clase. También tiene que invocar al constructor de la clase base.
- Método de nombre indiceSeguridad que calculará un índice de seguridad para el consumo de la seta, devolviendo un valor entero entre 0 y 100, donde:

```
0 → seta totalmente peligrosa
100 → seta completamente segura
```



Criterio de cálculo

El índice parte de un valor inicial de 100 puntos, y se irá reduciendo según los siguientes factores:

- El peso de la seta influye negativamente en la seguridad. A mayor peso, mayor reducción. Se descuenta el (peso / 10).
- El nivel de sabor indica la calidad de la seta. Cuanto menor sea el sabor, mayor será la penalización: se descuenta $(10 - \text{nivelSabor}) \times 5$
- Los días de conservación reducen la seguridad progresivamente: se descuentan 3 puntos por cada día.
- Si la seta es venenosa leve, se aplicará una penalización adicional fija de 25 puntos.

- Método de nombre `visualizaSeta()`, que visualice información de la seta con el formato:

Nombre: Niscalo - Especie: *Lactarius deliciosus* - Peso: 28.0 g - Sabor: 2 - Venenosa leve: true - Días: 5 - Seguridad: 17

- 3) Hacer un programa en el que se declare un objeto del tipo `SetaComestible`, al que se le asignen valores de nombre, especie y peso, leídos por teclado. Visualizar los datos del objeto. Modificar a continuación los valores de peso y `nivelSabor` y volver a visualizar los datos del objeto.

Ejemplo de ejecución del programa:

Introduce el nombre: Niscalo

Introduce la especie: *Lactarius deliciosus*

Introduce el peso: 28

--- DATOS INICIALES ---

Nombre: Niscalo - Especie: *Lactarius deliciosus* - Peso: 28.0 g - Sabor: 2 - Venenosa leve: true - Días: 5 - Seguridad: 17

Introduce el nuevo peso: 12

Introduce el nuevo nivel de sabor: 1

--- DATOS MODIFICADOS ---

Nombre: Niscalo - Especie: *Lactarius deliciosus* - Peso: 12.0 g - Sabor: 1 - Venenosa leve: true - Días: 5 - Seguridad: 13

- 4) En una carrera popular se desea gestionar los dorsales de los participantes utilizando un `ArrayList`.
- Declarar un `ArrayList` de nombre dorsales con datos de tipo entero, y asignar aleatoriamente 30 valores comprendidos entre 1 y 99, ambos incluidos, sin que haya dorsales repetidos. Visualizar a continuación el contenido del `ArrayList`.
 - Crear un segundo `ArrayList` llamado premiados. Añadir a premiados todos los dorsales que sean múltiplos de 5 o terminen en 7. Visualizar el contenido de premiados.
 - Eliminar del `ArrayList` dorsales todos los dorsales menores de 20. Visualizar nuevamente el contenido del `ArrayList`.

Puntuación de cada ejercicio:

Ejercicio 1:	1,5 puntos
Ejercicio 2:	3,5 puntos
Ejercicio 3:	2 puntos
Ejercicio 4:	3 puntos



Criterios de corrección y cómo entregar el examen:

Aunque un programa no funcione, entregarlo igual, ya que puede ocurrir que esté bien planteado y falle por un pequeño detalle. De la misma manera, si se pide hacer un programa con una determinada sentencia para implementar un bucle y no os sale, hacerlo de otra manera. Evidentemente no puntuará lo mismo, pero al menos podéis completar el ejercicio.

Hay que subir la solución del examen como tarea, en formato de procesador de texto en un único documento. El documento tiene que incluir el código fuente completo de cada uno de los ejercicios. Nombrar al documento con vuestro apellido